EXP NO: 9	*MODEL DEPLOYMENT: REST API WITH FLASK AND CONTAINERIZATION WITH DOCKER*

AIM:

To deploy a trained machine learning model as a REST API using Flask and to containerize the application using Docker so that it can run on any system.

ALGORITHM:

1. Import the required libraries (flask, joblib, requests, scikit-learn).
2. Train a Random Forest classifier on the Iris dataset and save it using joblib.dump().
3. Create a Flask application (app.py) with a GET route "/" for a health check and a POST route "/predict" that returns the species and confidence as JSON; invalid input returns an error with status code 400.
4. Create requirements.txt listing the dependencies.
5. Create a Dockerfile (base image python:3.11-slim) that installs the requirements, copies app.py and the saved model, exposes port 5000 and starts the app.
6. Run the API and test it by sending GET and POST requests using the requests library.
7. Build the Docker image using docker build and run the container using docker run, mapping port 5000.

CODE:

```
# 1. Import libraries
import os, sys, time, shutil, subprocess
import joblib, requests, sklearn
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


# 2. Train a model and save it to disk
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("Model accuracy:", accuracy_score(y_test, model.predict(X_test)))
joblib.dump(model, "iris_model.pkl")
print("Model saved as iris_model.pkl")
# 3. Create the Flask REST API (saved as app.py)
```

```
app_code = '''from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load("iris_model.pkl")
class_names = ["setosa", "versicolor", "virginica"]

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "Iris Prediction API is running",
                    "endpoints": {"GET /": "health check", "POST /predict":
                        "get prediction"}})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    if not data or "features" not in data or len(data["features"]) != 4:
        return jsonify({"error": "Send JSON like {'features': [sl, sw, pl,
            pw]}"}), 400
    features = [data["features"]]
    pred = int(model.predict(features)[0])
    proba = model.predict_proba(features)[0]
    return jsonify({"prediction": pred, "species": class_names[pred],
                    "confidence": round(float(proba[pred]), 4)})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
'''
open("app.py", "w").write(app_code)

# 4. Create requirements.txt and Dockerfile for containerization
open("requirements.txt",
    "w").write(f"flask\nscikit-learn=={sklearn.__version__}\njoblib\n")
dockerfile = '''FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py iris_model.pkl ./
EXPOSE 5000
CMD ["python", "app.py"]
'''
open("Dockerfile", "w").write(dockerfile)
print("\n----- Dockerfile -----\n" + dockerfile + "----- requirements.txt

    -----\n" + open("requirements.txt").read())

# 5. Run the API and test it with HTTP requests
server = subprocess.Popen([sys.executable, "app.py"],
    stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
time.sleep(3)
print("Server started at http://127.0.0.1:5000\n")
r = requests.get("http://127.0.0.1:5000/")
print("GET / ->", r.status_code, r.json())
samples = {"Setosa-like": [5.1, 3.5, 1.4, 0.2],
           "Versicolor-like": [6.0, 2.9, 4.5, 1.5],
           "Virginica-like": [6.9, 3.1, 5.4, 2.1]}
```

```
for name, feats in samples.items():
    r = requests.post("http://127.0.0.1:5000/predict", json={"features": feats})
    print(f"POST /predict {feats} ->", r.status_code, r.json())
r = requests.post("http://127.0.0.1:5000/predict", json={"features": [1, 2]})
print("POST /predict (bad input) ->", r.status_code, r.json())
server.terminate(); server.wait()
print("Server stopped.")

# 6. Build and run the Docker container
commands = ["docker build -t iris-api .",
            "docker run -d -p 5000:5000 --name iris-container iris-api",
            "curl -X POST http://localhost:5000/predict -H 'Content-Type:
                application/json' "
            "-d '{\"features\": [6.0, 2.9, 4.5, 1.5]}'"]
print("\nDocker commands:")
for c in commands:
    print("  $", c)
if shutil.which("docker"):
    print(subprocess.run(commands[0].split(), capture_output=True,
        text=True).stdout[-300:])
else:
    print("\nNote: Docker is not installed in this environment, so the image
        was not built here.")
    print("Run the three commands above on a machine with Docker installed.")
```

OUTPUT:

```
Model accuracy: 1.0
Model saved as iris_model.pkl


----- Dockerfile -----
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py iris_model.pkl ./
EXPOSE 5000
CMD ["python", "app.py"]
----- requirements.txt -----
flask
scikit-learn==1.8.0
joblib


Server started at http://127.0.0.1:5000

GET /  -> 200 {'endpoints': {'GET /': 'health check', 'POST /predict': 'get
    prediction'}, 'message': 'Iris Prediction API is running'}
POST /predict [5.1, 3.5, 1.4, 0.2] -> 200 {'confidence': 1.0, 'prediction': 0,
    'species': 'setosa'}
POST /predict [6.0, 2.9, 4.5, 1.5] -> 200 {'confidence': 1.0, 'prediction': 1,
    'species': 'versicolor'}
POST /predict [6.9, 3.1, 5.4, 2.1] -> 200 {'confidence': 0.99, 'prediction': 2,
    'species': 'virginica'}
POST /predict (bad input) -> 400 {'error': "Send JSON like {'features': [sl,
    sw, pl, pw]}"}
Server stopped.

Docker commands:
  $ docker build -t iris-api .
  $ docker run -d -p 5000:5000 --name iris-container iris-api
  $ curl -X POST http://localhost:5000/predict -H 'Content-Type:
      application/json' -d '{"features": [6.0, 2.9, 4.5, 1.5]}'

Note: Docker is not installed in this environment, so the image was not built
    here.
Run the three commands above on a machine with Docker installed.
```

RESULT:

The trained model was successfully deployed as a REST API using Flask; it returned the correct species with status code 200 for valid inputs and an error message with status code 400 for invalid input. The Dockerfile and requirements.txt were created for containerization. The docker build and docker run commands are listed above; Docker was not available in the environment used to run this program, so the image build was not executed here.